Migration Accelerator: Features and Where They Can Fail

Oct 26, 2026 (updated the same day, after commit `c81a148`)

Summary

As of now, the accelerator has about 55 working features in nine groups. They take a Synapse workspace from connection, through discovery, consultation, planning and migration, to validation in Fabric. Each feature below says what it does and where it can fail.

The failure points come from three places: limits the README documents, error paths in the code, and engineering judgment about live tenants. Judgment-based points are marked **(likely)**.

The five riskiest areas, where a first real client run is most likely to break:

1. **Table data load and sync pipelines.** Their Copy, Lookup and Script settings have only been tested offline against documented formats. The README says they must be confirmed on a first real run.
2. **Lakehouse target.** It was added on 23 October and is the newest code path. It converts T-SQL procedures into Spark notebooks, which is the hardest conversion in the tool.
3. **Storage routing through OneLake.** It rewrites storage paths inside code. Paths built in ways the patterns don't recognise stay unrouted and fail at run time.
4. **Connections and credentials.** Synapse never gives up its secrets, so every linked service needs a person to choose how Fabric signs in. Only SQL, ADLS Gen2 and Blob are converted.
5. **State held in memory.** Discovery, the run record and the agent live in the API process or the browser tab. A restart or a closed tab means redoing steps, although nothing in Fabric is lost.

The features added on 26 October (reconciliation, benchmark, access checks, capacity, cutover, Environment libraries, row-level security, the estate discovery) are tested offline only. Each one notes what must be confirmed on a first real run.

1. Connections and sources

Route picker

What it does: two dropdowns choose the source (From) and the destination (To). Only Azure Synapse to Microsoft Fabric is live. Eight other sources and four other destinations are shown dimmed as Coming soon, so the roadmap is visible.

Where it can fail:

- Customers may read the dimmed platforms as available. If a demo audience asks about Snowflake or SQL Server, the honest answer is that there is no converter for them yet.

Synapse sign-in and workspace selection

What it does: *Sign in with Azure*, using the Azure CLI or a Microsoft sign-in window, searches every tenant the account belongs to. The person then picks the subscription, resource group, workspace and dedicated SQL pool from lists, so nothing is typed. Connecting runs real checks on ARM access, data-plane access and the SQL pool.

Where it can fail:

- **Two separate permissions.** ARM needs Reader on the resource group; artifacts need a Synapse role such as Synapse Artifact User. Having one doesn't grant the other, so a client can pass one check and fail the other.
- **SQL catalog access** needs a database user that can read the catalog views. Without it, tables, views and procedures are missing.
- **Networking (likely):** a workspace behind a private endpoint or firewall rejects calls from the consultant's machine. The tool doesn't manage networks.
- **ODBC Driver 18** must be installed on the machine running the API, or no SQL can be read.

- **The Synapse sign-in isn't kept across restarts.** After an API restart a person must sign in again to rediscover, load data or count rows.

Git repository source (recommended)

What it does: when the Synapse workspace is Git-integrated, the tool reads the repository, collaboration branch and root folder from Azure and clones it locally with `git`. Definitions come from the repository, which includes work that is merged but not yet published, and the commit is recorded (for example `main · 3f9a1c2`). Any branch can be chosen under More options. A repository URL can also be typed.

Where it can fail:

- **Git credentials:** the clone uses the machine's own credential helper or SSH key. If nobody has cloned the repository once in a terminal, the clone fails. A URL with credentials in it is refused on purpose.
- **Repository and workspace differ:** Spark pools and libraries aren't in a repository. They appear only when the workspace itself is signed in to.
- **Unusual folder layouts (likely)** or a wrong root folder give an empty or partial inventory.

Live workspace source

What it does: reads what is published in the workspace through the Synapse Artifacts API, using GET calls only.

Where it can fail:

- It misses work that is merged in Git but not published.
- A 403 on one artifact type removes that type from the counts and reports an issue. A person must notice the issue, or that type is quietly not migrated.
- **Throttling (likely):** very large workspaces may hit API rate limits during discovery.

Workspace export (ZIP)

What it does: accepts a ZIP of the repository folder, up to 200 MB. It is extracted safely, with no paths outside its folder, no symbolic links, and limits on size and file count. The folder holding `pipeline/`, `linkedService/` and the others is found automatically. A dedicated SQL pool can still be read from Azure.

Where it can fail:

- Exports over 200 MB, or with more files than the limit, are refused.
- Uploads are kept only while the API process that received them is running. A restart deletes them, so the ZIP must be uploaded again.
- Without *Also read the dedicated SQL pool*, tables, data loads and row counts are skipped.

Environment parameters file

What it does: an optional `TemplateParametersForWorkspace.json` replaces the linked-service values committed in Git, which are usually Dev's, with Test or Prod values. Secrets in the file are masked or left out and never reach the disk.

Where it can fail:

- Only parameters named `<linked service>_connectionString` or `<linked service>_properties_typeProperties_<property>` are applied. Other naming styles are listed as unmatched, not guessed.
- Because secrets are dropped, every Fabric connection still needs its credentials typed in Plan.

Fabric connection and capacity check

What it does: signs in with the Azure CLI or Fabric CLI, lists the workspaces, and checks that the chosen one has a Fabric capacity. Start migration stays disabled until both sides pass, with a checklist of what is missing.

Where it can fail:

- **No capacity or a paused capacity:** a run is refused up front.

- **Fabric CLI route:** the Fabric CLI can't issue a SQL token, so the first SQL object opens a Microsoft sign-in window. That window fails when the API runs hidden as the background service.
- **Different accounts on each side (likely):** signing in to Synapse and Fabric as different users leads to confusing permission errors later.

2. Discovery

Discovery engine

What it does: reads every object from up to three sources: the Git repository, the live workspace, and the dedicated SQL pool catalog through 22 fixed SELECT queries. It joins what each source says about the same object into one record, and notes where Git and the published workspace disagree (drift). It covers SQL pools, tables, views, procedures, functions, scripts, pipelines, datasets, linked services, triggers, notebooks, Spark pools, Spark jobs, libraries, integration runtimes, security, data masks and external tables. It also reads the pool's **estate**: rows and size per table, row-level security policies, external data sources and file formats, and recent query history with peak concurrency. Lake databases come from the live workspace or the repository's `database` folder. Notebooks that run other notebooks (`%run`, `notebook.run`) by a written-out name become dependencies. It never writes and uses no AI. It also scans definitions for embedded secrets. The same engine runs from the command line (`python -m discovery_agent`).

Where it can fail:

- **Not discovered at all:** networking (managed private endpoints, firewall). It is absent from the plan unless someone knows to look.
- **Sizes and query history need** `VIEW DATABASE STATE`**.** Without it they are reported as unknown, and the capacity assessment and benchmark have less to work from.
- **Lake databases from the live workspace** use a data-plane route not yet confirmed on a real tenant. If it fails, discovery says *could not be listed*; the repository route doesn't depend on it.
- **Replicated tables' sizes (likely)** are taken from one compute node's copy, which may not match what the pool reports elsewhere.
- **Serverless SQL** is out of scope. Scripts and pipelines written for it are only flagged.
- **Partial reads:** an endpoint that returns 403 is reported as an issue, not a count. The inventory is incomplete until a person fixes the permission and runs discovery again.
- **Parsing edge cases (likely):** dynamic SQL, objects created at run time, and references built from strings can hide dependencies from the parser.
- **Results live in memory:** the last discovery is saved to `.accelerator/state`, but if that is turned off or the state is lost, discovery must run again.

Inventory, overview and export

What it does: the Overview tab shows one tile per object type, and clicking a tile filters. The Inventory tab is a searchable table, and clicking a row shows the object's definition, Fabric target and the steps it needs. Export in the top bar downloads the whole inventory as JSON.

Where it can fail:

- **Large estates (likely):** very large inventories, tens of thousands of objects, may make the browser table slow. The demo uses about 1,500 objects.
- The JSON export holds object definitions. Treat it as client-confidential.

Workspace explorer

What it does: shows the discovered workspace as a repository tree in the folders Synapse Git uses, with the SQL pool nested by schema. Everything else goes under *other*, so every object appears once and the total matches Discover. It has a name filter, and clicking an object opens its details.

Where it can fail:

- It shows only what discovery found, so every discovery gap above shows up here too.
- Below 1,100 px wide it moves above the steps, which takes space on small laptops.

3. Assessment and dependency waves

Classification and component mapping

What it does: puts every object into one of five classes:

- **Direct:** Fabric has a matching component.
- **Reconfigure:** the same idea, set up differently.
- **Transform:** the code or settings change.
- **Manual:** a person has to do it.
- **Review:** there is no reliable one-to-one mapping.

There are 24 type rules. Each object also gets a workstream and a Synapse-to-Fabric component mapping, for example dedicated SQL pool to Warehouse, or linked service to Connection.

Where it can fail:

- **Class by type, not content (likely):** a procedure classed as Transform may be trivial or very hard. The class alone doesn't show difficulty; the planner's risks do.
- **Unknown types** fall to Review, so new Synapse features aren't handled until a rule is added.

Dependency graph and waves

What it does: builds a graph of what depends on what: a view on its tables, a pipeline on its notebooks and connections. It groups objects into waves so nothing is created before what it needs. Waves can be viewed as a list or a graph, and wave 1 doubles as a pilot scope.

Where it can fail:

- **Hidden dependencies:** references in dynamic SQL, notebook code that builds table or notebook names from strings, or parameters resolved at run time aren't seen. Notebooks run by a written-out name are now ordered correctly. The object can land in an earlier wave than its dependency and fail with "object not found". Retry failed usually fixes it once the dependency exists.
- **Circular references (likely):** two objects that refer to each other can't be ordered cleanly, and one will fail on the first pass.
- Cross-workspace dependencies, such as a pipeline calling another workspace, aren't modelled.

4. Consultation and agent orchestrator

Consultation and approved brief

What it does: asks the questions a consultant would agree with the client before anything moves. They are grouped into ten topics: scope, target architecture, data loading, SQL, Spark, pipelines, security, validation and cutover, special features, and context. Three rules keep it grounded:

- A question appears only when discovery found that feature.
- Settings the configuration already knows, such as the Fabric workspace and capacity, are shown rather than asked.
- Every option is something Fabric and the tool can actually do.

Each question has a recommendation with a reason, and is labelled by its effect: it drives the run, it makes the agent stop for a person, or it is only recorded. The answers become a versioned brief that a person submits and approves. A change after approval is flagged.

Where it can fail:

- **Some answers are still recorded only.** Reconciliation tolerance, view and procedure comparison and the SLA now drive Validate, and 17 answers (downtime, formal rollback, schema freeze, dynamic SQL review, libraries and others) become cutover sign-off items the agent waits for. Big bang vs phased, priority and naming standards are still only written into the brief.
- **Sign-offs are a person's word.** The agent can't check that a benchmark was run or a rollback plan written; it waits until someone confirms it.
- **Recommendations come from discovery facts.** If discovery missed something (a lake database, say), no question is asked about it.
- **Stored in the browser (likely):** the brief lives in the browser per project. Another machine or a cleared browser loses it.

Agent orchestrator

What it does: carries out the approved brief in nine stages: the approved brief, a discovery snapshot, dependency analysis, the migration plan, a readiness check, approval, the migration run, validation, and a cutover sign-off that lists what the brief says a person confirms before users move to Fabric. It uses exactly the same actions a person uses in Plan, Migrate and Validate, so it has no hidden powers. It stops for a person when the brief, a risk or a failure asks for one, showing the reason and the choices. Finished stages confirm the matching steps. Everything it does is logged in the activity panel.

Where it can fail:

- **It runs in the browser tab.** Closing the tab stops the agent; a run already started in Fabric keeps going. Resume continues from the first stage that isn't done.
- **It never rediscovers on its own.** If the Synapse estate changes after discovery, the agent works from the stale snapshot.
- **Gates depend on the brief.** If the brief doesn't ask for approval before writing, the agent goes straight from readiness to the run.
- **New code (likely):** the orchestrator was added on 22 October. Edge cases such as a failure during a retry, or the brief changing mid-run, have had little real use.

5. Plan

Deterministic planner

What it does: gives every object a strategy: Automated, Manual, Assess first, Later or Not selected. It reads the real definitions and runs about 30 pre-flight checks, grading each finding as Blocking, High, Medium or Low. It estimates effort in hours and days per object, wave and type, and scores readiness from 0 to 100. The score is 100 minus a penalty per finding: 25 for Blocking, 6 for High, 2 for Medium and 0.5 for Low. A Blocking risk stops Plan from being confirmed. The backend checks again when a run starts and refuses it (`409 plan_blocked`) before touching Fabric.

Where it can fail:

- **The score is a heuristic.** Effort figures and penalties are fixed weights, not calibrated against real projects yet. Treat them as relative, not as a quote.
- **The checks only know the patterns they look for.** T-SQL that Fabric rejects but no check catches is found only at run time.
- **A wrongly Blocking finding** stops the whole plan. The only way past it is to take the object out of scope.

Stages, options and credentials

What it does: each of the 11 stages is switched on or off and tuned here. Options include the target (Warehouse or Lakehouse), the Warehouse collation, how table data loads (create and run, or create only), skip or replace tables that have rows, date filters and sync, external storage routing, committing to Fabric Git, and updating objects already in Fabric. Credentials that Synapse won't give up are typed here: data-load connection, linked services and storage. They are sent once when the run starts, held in memory, and never stored, logged or returned.

Workspace identity and Key Vault references are offered first, so no secret passes through the tool.

Where it can fail:

- **Collation is fixed at creation.** A wrong choice means dropping and recreating the Warehouse.
- **Typed credentials are lost** when the API restarts, so a retry after a restart needs them typed again.
- **Update existing** overwrites changes made by hand in Fabric since the last run.

Capacity assessment

What it does: Plan → Capacity gives a Fabric SKU starting point with its evidence: the SQL pool's DWU, the largest Spark pool's vCores (1 CU = 2 Spark vCores, as documented), the data the first load moves, the peak number of SQL requests running at once, and the busiest hour of schedule triggers. It shows the SKU for the larger workload and the SKU for SQL and Spark at once, and whether the target workspace's capacity covers it.

Where it can fail:

- **The SQL figure rests on an assumption.** Microsoft publishes no DWU-to-CU conversion, so it uses 6 CU per DW100c and says so. Calibrate it with a pilot and the Fabric Capacity Metrics app.
- **Missing inputs:** without `VIEW DATABASE STATE`, data volume and concurrency are unknown and the estimate rests on pool sizes alone.
- **Trial and P SKUs** can't be compared and are reported as such.

6. Migrate

Warehouse, schemas and SQL objects

What it does: creates one Warehouse per SQL pool, with its schemas and empty tables. Types are mapped, keys are recreated as `NOT ENFORCED`, IDENTITY becomes `bigint IDENTITY`, and dynamic data masks go into the `CREATE TABLE`. It then creates scalar and inline table-valued functions, views and procedures after seven families of T-SQL rules. The rules strip distribution, index and partition clauses, turn `RENAME OBJECT` into `sp_rename`, and remove isolation-level and workload-management statements. They work inside dynamic SQL and never touch strings or comments. Removed statements stay as comments.

Where it can fail:

- **T-SQL that Fabric doesn't support** and the rules don't rewrite: `MERGE` in some forms, `SET ROWCOUNT`, `FOR XML`, synonyms, `PREDICT`, triggers, materialized views, `sys.pdw_*` views and multi-statement table-valued functions. Most are flagged in Plan; the rest fail at create time with Fabric's error.
- **IDENTITY behaves differently in Fabric:** only `bigint`, with no custom seed or increment. Code that relies on specific new key values may behave differently after migration.
- **Unsupported column types** are mapped and noted. A masking function Fabric lacks stops the table before any data loads.

Lakehouse target (new)

What it does: added on 23 October. The SQL pool can go to a Lakehouse instead of a Warehouse. Delta tables are created by one Spark notebook. Views, functions and permissions go on the SQL analytics endpoint. Stored procedures and scripts that write data are converted from T-SQL to Spark notebooks, and pipelines run those notebooks. Data loads into Delta tables, and validation reads the Lakehouse.

Where it can fail:

- **Converting procedures to Spark is the hardest conversion in the tool.** Transactions, cursors, temp tables, `@@ROWCOUNT` logic and error handling have no direct Spark equivalent. Expect many objects to need AI drafts or hand work.
- **The SQL analytics endpoint is read-only,** so anything that writes through SQL must become a notebook.
- **Newest code path (likely):** it has had the least real use of any stage.

Roles and permissions

What it does: recreates custom roles, adds role members, and replays GRANT and DENY at database, schema, object and column level, keeping `WITH GRANT OPTION`. A Microsoft Entra user is created by its first GRANT, because Fabric has no `CREATE USER`. **Row-level security** policies are recreated with their FILTER predicates and their on/off state, after the predicate functions they call; Validate checks them.

Where it can fail:

- **SQL-authenticated users can't move.** Fabric has no SQL authentication, so Plan lists the Entra principal a person must choose.
- **Entra users from another tenant (likely),** or ones without access to the Fabric workspace, make the GRANT fail.
- Workspace roles still decide who can connect. A GRANT alone doesn't give a user access to the workspace.
- **BLOCK predicates** (rules that stop writes) aren't carried: Fabric Warehouse has FILTER predicates only. They are listed for a person.
- **Row-level security covers T-SQL only.** Workspace Admins, Members and Contributors still see every row through Spark and OneLake.

Table data load

What it does: one metadata-driven Fabric pipeline per wave, `load_<warehouse>_wave_<n>`. A ForEach runs a Copy activity that reads the pool with a typed SELECT and writes the Warehouse with COPY and staging. IDENTITY tables go through a staging table, then `IDENTITY_INSERT` and a reseed, so key values are kept. It can create and run the pipeline (the default) or only create it, and can skip or replace tables that already have rows. Row counts are compared after the load. Data never passes through the tool.

Where it can fail:

- **Not yet confirmed on a real run.** The README says the Copy settings (Synapse source, Warehouse sink with staging) must be confirmed on a first real run. This is the single biggest open risk.
- **Connection to the pool:** it needs a Fabric SQL connection that can reach Synapse. Firewalls, private endpoints or a missing credential make the Copy fail.
- **Large tables (likely):** very large tables may hit pipeline timeouts or capacity throttling on small Fabric SKUs.
- **Row counts need the Synapse connection.** Without it, counts are reported as unread.

Date filters and keeping tables in sync

What it does: a table can load only rows in a fixed date range, then keep in sync. The sync uses a change column such as `ModifiedAt` and a watermark. A control table `migration_sync.sync_tables` holds each table's watermark and query. One `sync_<warehouse>` pipeline is created with a daily schedule at 02:00 UTC, switched off. Each sync deletes and inserts by key in one transaction. A re-read window of one hour or one day catches rows committed late.

Where it can fail:

- **Hard deletes** in Synapse are never carried over.
- A change column the source doesn't always update misses those changes.
- **Tables without keys** are appended, so updated rows appear twice.
- A load while the sync schedule is on can lose or duplicate rows. The schedule must be off during a reload.
- **Not yet confirmed on a real run:** the sync pipeline's Lookup, Copy and Script activities are in the same position as the data load.

Spark pools and Environments

What it does: creates a custom Spark pool and a published Environment, mapping node size, autoscale and runtime. The pool's requirements.txt or environment.yml becomes the Environment's public libraries, and its Spark configuration becomes the Environment's Spark properties.

Where it can fail:

- **Workspace packages aren't copied.** Wheel and jar files uploaded to Synapse stay there; they are named for a person to upload.
- **The library upload isn't confirmed on a real run.** It uses the Environment definition format. If Fabric refuses it, the Environment is still created and a note says to add the libraries by hand.
- **Requirements options** (`-r`, `--index-url`, URLs, local paths) are named, not carried.
- Node sizes or runtimes with no Fabric equivalent are mapped to the nearest one, which can change performance.

Notebooks

What it does: keeps the cells, drops the outputs, and attaches each notebook to the Environment made from its Spark pool. It rewrites `mssparkutils` to `notebookutils`, points Synapse SQL connector reads at Fabric's connector and the Warehouse, and points `%run /folder/name` and `notebook.run("/folder/name")` at the notebook's name, as Fabric runs it. Other Synapse-only calls are flagged: `TokenLibrary`, `mssparkutils.env`, connector writes, ADLS paths, and notebook names built at run time. Plan flags a call to a notebook discovery never found.

Where it can fail:

- **The Environment attachment isn't confirmed on a real run.** If the Spark pool wasn't migrated first, the notebook runs on the workspace's default settings, and the note says so.
- **Two notebooks with the same name in different Synapse folders (likely)** collide, because every notebook lands at the workspace root and is run by name.
- **.NET (C#) notebooks** are deferred.
- **Rewrites are pattern-based (likely):** aliased imports or wrapper functions around `mssparkutils` may be missed.

Connections from linked services

What it does: turns SQL, ADLS Gen2 and Blob linked services into Fabric connections, using credentials chosen in Plan. A parameterised linked service uses its parameters' default values.

Where it can fail:

- **Every other linked-service type** (REST, Oracle, SAP, Snowflake and so on) is created by hand.
- **On-premises sources** behind a self-hosted integration runtime need a Fabric gateway, which is always set up by hand.
- **Default parameter values** may point at Dev when the target is Prod.

Pipelines and datasets

What it does: recreates each pipeline with its datasets embedded in the activities. Notebook, pipeline and Spark job references become Fabric ids, and reads of the Synapse pool point at the Warehouse. 21 activity types have mapping rules.

Where it can fail:

- **A pipeline with a mapping data flow,** or any activity with no Fabric equivalent, is not created at all. The activity names are listed.
- **Expressions (likely):** Synapse expressions that reference workspace-level system variables or linked-service parameters may resolve differently in Fabric.

Spark job definitions and SQL scripts

What it does: Spark job definitions keep their main file, class, arguments and libraries, attached to the Environment of the same name. SQL scripts become a notebook with a T-SQL cell bound to the Warehouse, after the T-SQL rules.

Where it can fail:

- A job's main file in storage that Fabric can't reach fails at run time unless storage routing covers it.
- **Scripts written for serverless SQL** are flagged as `SERVERLESS_SQL` and should be left out.

Schedules

What it does: schedule triggers become pipeline schedules, created switched off, for minute, hourly, daily and weekly recurrences.

Where it can fail:

- Event and tumbling-window triggers, every-N-days and monthly schedules are recreated by hand.
- Schedules start switched off. Validate → Cutover switches them on, and it's listed in the agent's sign-off; until then nothing runs in Fabric.

External tables and storage routing through OneLake

What it does: external tables become OneLake shortcuts in a Lakehouse. Code that reads storage directly (`abfss://` or `https://...dfs/blob`) is listed under External storage. Each routed location gets a shortcut under `Files`, and every path to it is rewritten: literal paths, f-strings, and T-SQL that builds the host. Spark jobs run a rewritten copy of their main file.

Where it can fail:

- **Paths the patterns don't recognise:** paths read from config tables, parameters, environment variables or built across several lines stay unrouted. Plan marks such objects `EXTERNAL_STORAGE`, and they fail at run time.
- **Cross-tenant storage** needs an account key or SAS connection. The signed-in user's identity is rejected.
- **A shortcut whose connection can't read the storage** fails everything that reads through it. Validate's access check now finds this before cutover.

Run control

What it does: pre-run checks, then the run wave by wave. You can pause, resume, Retry failed, and stop at the end of a wave with failures. Objects already in Fabric are skipped. With *Update objects that are already in Fabric*, views and procedures use `CREATE OR ALTER` and items keep their id. Tables are never replaced.

Where it can fail:

- **No rollback of created items.** Nothing created in Fabric is deleted by the tool, and Reset doesn't remove Fabric items. The rollback lever is at cutover: switch schedules off, and point consumers back at Synapse.
- **A skipped table keeps its old structure.** Schema changes made in Synapse after the first run need a manual drop or a new table.
- **The run record lives in the API's memory.** After a restart the run shows *Run again*, though Fabric still holds what was created.

7. AI-assisted conversion and Fabric Git

AI-assisted conversion (LLM transforms)

What it does: for objects the rules can't convert, a person can ask Claude for a draft. Supported types are procedures, views, tables, SQL scripts, pipelines and notebooks, with separate prompts for the Warehouse and Lakehouse targets. The API runs the Claude Code CLI headless, with no tools, no MCP servers and an empty folder, so it can only answer, and no API key is needed.

The review screen shows the exact prompt and a three-way compare: Synapse source, rule output and AI draft. Automatic checks run on the draft, and a draft that fails them can't be accepted. An accepted draft is pinned to a hash of its source; if the source changes, the run falls back to the rules. A per-project `project.md` holds the client's coding standards. Drafts run a few at a time in the background.

Where it can fail:

- **Claude Code must be installed and signed in** on the API machine, or generation is unavailable. Setting `MIGRATION_LLM=off` disables it.
- **Client code leaves the network.** Definitions, rule output, dependency names and column types are sent to Anthropic. Some clients won't allow that.
- **Drafts can be wrong in ways the checks don't catch.** The checks test structure, not business logic, so the human review is the real safeguard.
- **Timeouts:** each draft has 180 seconds by default. Very long procedures or notebooks may time out.
- **Cost and speed (likely):** two drafts run at a time by default, so hundreds of objects take a long time.

Fabric Git commit

What it does: when the Fabric workspace is connected to Git, each completed run makes a selective commit to its `dev` branch. The commit contains only the items this run created or updated, and the screen offers **Open pull request dev → main**. Data-load and sync pipelines, deletions, and changes the run didn't make are left out and named. A commit problem never fails the run, and **Commit now** retries it. Git credentials are set to Automatic for Azure DevOps, so no token is entered.

Where it can fail:

- **It needs setup by hand first:** the Fabric repository and branches, the tenant setting for Git sync, and connecting the Dev workspace to Git. Otherwise it reports *Not connected*.
- **It stops rather than forcing** when the branch has changes the workspace hasn't taken, or when there are conflicts. A person resolves these in Fabric's Source control panel.
- **Some item types' Git support is still in preview in Fabric,** for example the Warehouse. That limit is Fabric's, not ours.
- **GitHub repositories** need the GitHub account connected once in Fabric.
- **Deploying to Test and Prod** with `fabric-cicd` isn't built yet.

8. Validate

Object-by-object comparison

What it does: reads both sides without writing and compares them category by category:

- The Warehouse and its schemas exist.
- Table columns, types and data masks match.
- Row counts match, taking date filters into account. A synced table counts as in step when its count lies between the last sync and the current filter.
- Function, view and procedure definitions match, ignoring comments and whitespace.
- Roles, members and GRANT or DENY states match.
- Spark pool and Environment settings match.
- Notebooks have the same cell counts, and pipelines the same activity counts.
- Connections, jobs, scripts, schedules and shortcuts exist.

A check that couldn't run is shown as Review with the reason, never as a match.

Where it can fail:

- **Definition text comparison** flags every object the rules or AI changed as different, so many Review rows are expected. The value and result checks below are what prove them.
- **Row counts need the Synapse connection.** After a restart without signing in again, they show as unread.

Data reconciliation and result equivalence

What it does: for each table whose row counts agree, one aggregate query per side reads a value profile: NULLs per column, exact sums of numeric columns, earliest and latest dates, total text length, and duplicate keys. The profiles are compared exactly or within an approved tolerance (0.01%, 0.1% or 1%), and a difference is a

mismatch or left for a person. Views can be compared the same way, by what they return. Procedures are compared by their first result set's columns and types, described on both sides without running them. Each shows as its own row: Data Profile, View Results, Procedure Results.

Where it can fail:

- **It reads every table in full on both sides.** On very large pools this is slow and uses capacity; choose *row counts only* for those, or validate a pilot wave.
- **A profile is strong evidence, not proof.** Two different datasets could in theory share every measure.
- **Types the migration changes** (`datetimeoffset`, `time`, binary, spatial, `text`) are profiled for NULLs only.
- **Sum overflow (likely):** very large decimal columns can overflow `decimal(38, s)`; that table shows as Review with the error.
- **Emoji in text (likely)** can make `nvarchar` and UTF-8 `varchar` lengths differ slightly.

Storage access

What it does: every OneLake shortcut the run made is read through OneLake. Through a shortcut, storage is read with the shortcut's connection, so a missing permission (403) or a missing path (404) shows in Validate → Access rather than when a notebook runs.

Where it can fail:

- **It needs a OneLake sign-in** through the Azure CLI; without it the check reports *not checked*.
- **Key Vault access** for notebooks that read secrets isn't checked.

Benchmark against Synapse

What it does: the most demanding recent SELECTs from Synapse's request history are run in Fabric twice, and the warm run is compared with Synapse's recorded average. A slowdown beyond the allowed percentage is Review, or a mismatch when an SLA was agreed. Queries that write or use temporary tables or variables are never run.

Where it can fail:

- **The two times aren't measured the same way.** Synapse's is server-side; Fabric's is measured from the API machine and includes returning the rows, so queries with large results look slower than they are.
- **The history keeps only the pool's last 10,000 requests,** so weekly or monthly reports may not be in it.
- **Queries that reference other databases or session state** can fail in Fabric for reasons that aren't performance.

Runs-in-Fabric checks

What it does: proves objects work, not just that they exist. Tables and views are queried with `SELECT TOP (1)`. Procedures are compiled with `sp_describe_first_result_set`, or run inside a transaction that is rolled back when ticked. Pipelines, notebooks and Spark jobs are judged by their latest run, or run now, three at a time, within a time limit. An object that matches but has never run is shown as Review.

Where it can fail:

- **Procedures using temporary tables** can't be compiled ahead and are reported as not checked.
- **"Run now" makes real runs.** They read real sources and write real targets, including any external systems a pipeline touches.
- **Long-running jobs** time out and show as not finished rather than failed.

Cutover

What it does: Validate → Cutover shows whether Fabric is ready to take over: the last validation has no mismatches, every synced table is current (its watermark against the newest change in Synapse, with the lag), and the last sync succeeded. It offers three actions, each only when pressed: run the final sync now, switch the pipeline schedules on (or off again to roll back), and record who approved the cutover. Synapse is never touched; stopping

its triggers is listed for a person.

Where it can fail:

- **Not confirmed on a real run:** switching schedules on and off uses Fabric's schedule API, and the final sync runs the same sync pipeline that is itself unconfirmed.
- **The approval record lives in the API's memory.** A restart loses it; the agent's sign-off and the brief keep their own record in the browser.
- **Hard deletes** in Synapse still aren't synced, so a table can look current while holding rows Synapse deleted.

Final report

What it does: confirming Validate shows *Migration complete* with the final numbers, stated plainly if objects failed or checks didn't match. **Download report (CSV)** lists every object's result and every validation check.

Where it can fail:

- Validation results live in the page. Refreshing the browser before downloading means running validation again.

9. Platform features

Projects

What it does: the project menu creates, renames, switches and deletes projects. Each keeps its own plan, stage options, confirmations and progress.

Where it can fail: projects are stored in the browser. A different browser, another machine or cleared site data starts empty, and two consultants can't share a project.

Demo mode

What it does: the Live / Demo switch runs the whole app on a sample Synapse workspace (`demo-synapse-ws`, about 1,500 objects) and a sample Fabric workspace, with simulated results and canned AI drafts. No Azure access is needed. When the backend doesn't answer in Live mode, a bar offers **Use Demo data**.

Where it can fail: demo results are simulated, so a smooth demo says nothing about how a real tenant will behave. Set that expectation with the audience.

Saved state and Reset step

What it does: the last discovery and the Fabric target are saved to `.accelerator/state`, and come back after a restart; tokens and credentials are never saved. Every step has **Reset step**, which clears that step and everything after it, after listing exactly what will be cleared. When a confirmed step's results are lost, its box shows *Run again* and later steps wait.

Where it can fail:

- **The run record and validation results aren't saved.** A restart means redoing those steps, though nothing in Fabric is lost.
- Reset never deletes anything in Fabric, which surprises people who expect a clean slate.

Background API service

What it does: `install-api-service.ps1` creates the Windows scheduled task `MigrationAcceleratorAPI`. It runs the API hidden, with auto-reload, at every Windows sign-in, without admin rights, and logs to `.accelerator\api.log`. Running `start-ui.ps1` again is safe: what is running is reused, and a second copy on the same port is refused.

Where it can fail:

- **Interactive sign-in windows can't appear** from a hidden task, such as the Fabric CLI's SQL sign-in. Use the Azure CLI route instead.
- **Windows only.** Machine policies that block scheduled tasks or PowerShell scripts stop it.
- Auto-reload restarts the API when code changes, which clears in-memory runs and typed credentials.

Error references and redaction

What it does: an unexpected error never shows a stack trace in the browser. It is written, with secrets redacted, to the API log, and the screen quotes a short reference such as *reference 7f3a2c1b*.

Where it can fail: a person needs access to the API machine's log to look the reference up.

Activity panel

What it does: at the foot of every page, a newest-first log of what you, the agent, runs and the system did. It can be filtered to Problems, Agent, You, or Runs and jobs, and paused to read while work goes on.

Where it can fail: it lives in the page, so a refresh clears it. It shows the latest 150 entries.

Migration assistant panel (preview)

What it does: the **+** button or Ctrl+I opens a chat panel docked on the right. It shows the context it will use and suggested questions.

Where it can fail: the conversation isn't built yet, and nothing typed is sent anywhere. Don't present it as working.

Risk table

One row per feature, with its main failure point, sorted from most to least likely to cause trouble on a first real client run. The likelihood is engineering judgment, not measured: High means expect it on most real estates, Medium on some, Low rarely.

Feature	Main failure point	Likelihood
Table data load	Copy settings not yet confirmed on a real Fabric run	High
Date filters and sync	Sync pipeline not confirmed on a real run; hard deletes not carried	High
Lakehouse target	T-SQL procedures to Spark is hard; newest code	High
Connections from linked services	Only SQL, ADLS Gen2 and Blob converted; secrets typed by hand	High
Storage routing through OneLake	Paths built from config or parameters stay unrouted	High
Spark pools and Environments	Library upload not confirmed on a real run; workspace packages by hand	Medium
Notebooks	Environment attachment not confirmed on a real run	Medium
Pipelines and datasets	Pipelines with mapping data flows not created	Medium
Warehouse and SQL objects	T-SQL Fabric rejects that no rule or check covers	Medium
Dependency waves	Dynamic SQL hides dependencies; order is wrong on the first pass	Medium
Discovery engine	Networking not discovered; sizes and history need VIEW DATABASE STATE	Medium
Synapse sign-in	ARM and data-plane permissions differ; private networks	Medium
Roles and permissions	SQL users need an Entra replacement chosen by a person	Medium
AI-assisted conversion	Wrong logic passes the checks; client code leaves the network	Medium
Fabric Git commit	Needs one-time setup; stops on conflicts	Medium
Agent orchestrator	Stops when the tab closes; new code	Medium
Data reconciliation	Reads every table in full on both sides; slow on large pools	Medium
Cutover	Schedule switching and final sync not confirmed on a real run	Medium
Capacity assessment	SQL figure rests on an assumed DWU-to-CU rate	Medium
Benchmark	Fabric time includes returning rows; history is short	Medium
Planner and readiness score	Weights not calibrated on real projects	Medium
Saved state	Run and validation results lost on restart	Medium
Validation comparison	Definition text differs for every converted object	Low

Feature	Main failure point	Likelihood
Git repository source	Clone fails without a saved Git credential	Low
Workspace export (ZIP)	200 MB limit; deleted on restart	Low
Environment parameters file	Only two parameter naming styles applied	Low
Fabric connection and capacity	Fabric CLI SQL sign-in fails under the hidden service	Low
Schedules	Created off; event and tumbling triggers by hand	Low
Runs-in-Fabric checks	Procedures with temp tables not checked; Run now makes real runs	Low
Consultation	A few answers still recorded only; sign-offs are a person's word	Low
Storage access check	Needs a OneLake sign-in; Key Vault not checked	Low
Row-level security	BLOCK predicates by hand; T-SQL reads only	Low
Background service	Hidden task can't show sign-in windows	Low
Projects	Stored per browser; can't be shared	Low
Migration assistant panel	Conversation not built	Low

The most useful next step is one end-to-end run on a real Synapse workspace and Fabric capacity. That would confirm or rule out the top two rows, and the new cutover, Environment and reconciliation code with them.